

Systeme, Scripts et Sécurité

Votre machine virtuelle doit avoir comme nom de session :
La_Plateforme et comme mot de passe : LAPlateforme_.

```
“sudo adduser --allow-bad-names La_Plateforme”
```

on utilise la commande “--allow-bad-names” afin de pouvoir mettre une majuscule et un caractère spécial ce qui est de base pas autorisé sur linux.

Utilisez les lignes de commande pour créer cinq fichiers textes nommés “mon_texte.txt” et assurez-vous qu’ils contiennent le texte suivant :

« Que la force soit avec toi. » .

Répartissez ces fichiers dans les répertoires suivants : “Bureau”, “Documents”, “Téléchargement”, “Vidéos” et “Images”.

```
“mkdir -p ~/Bureau ~/Documents ~/Téléchargements ~/Vidéos ~/Images”
```

Commande pour s'assurer que tous les dossiers sont bien créés.

```
“echo "Que la force soit avec toi." > ~/Bureau/mon_texte.txt”
```

```
“echo "Que la force soit avec toi." > ~/Documents/mon_texte.txt”
```

```
“echo "Que la force soit avec toi." > ~/Téléchargements/mon_texte.txt”
```

```
“echo "Que la force soit avec toi " > ~/Vidéos/mon_texte.txt”
```

```
“echo "Que la force soit avec toi." > ~/Images/mon_texte.txt”
```

Commande pour créer les fichiers “txt” avec “Que la force soit avec toi” dedans, et les répartir grâce a ~/ dans les bons dossiers.

À partir du répertoire de votre session , utilisez le terminal et le mot “force” pour localiser les cinq fichiers “mon_texte.txt”.

`“grep -rl "force" ~”`

Commande pour localiser les fichiers “txt” que l’on vient de créer.



```
La_Plateforme@debian01: ~  
Fichier Édition Affichage Recherche Terminal Aide  
La_Plateforme@debian01:~$ grep -rl "force" ~  
/home/La_Plateforme/Téléchargements/mon_texte.txt  
/home/La_Plateforme/Documents/mon_texte.txt  
/home/La_Plateforme/Vidéos/mon_texte.txt  
/home/La_Plateforme/.bashrc  
/home/La_Plateforme/Images/mon_texte.txt  
La_Plateforme@debian01:~$
```

Créez un répertoire nommé "Plateforme" dans le dossier "Documents" de votre session et ajoutez-y le fichier “mon_texte.txt” précédemment créé.

`“mkdir -p ~/Documents/Plateforme”`

Commande pour créer le dossier Plateforme dans mes documents.

`“cp ~/Documents/mon_texte.txt ~/Documents/Plateforme/”`

Commande pour copier le fichier “mon_texte.txt” de mes documents dans mon dossier “Plateforme”

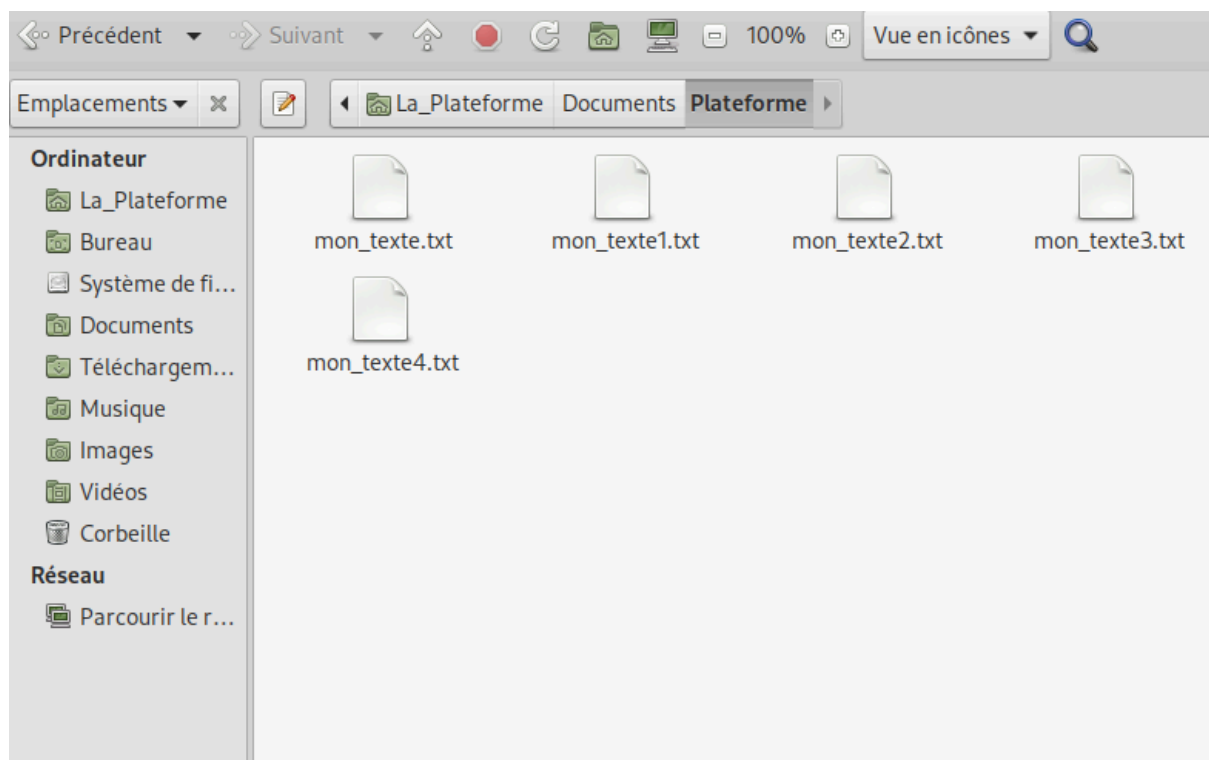
Dupliquer ce fichier quatre fois dans le même répertoire, formant ainsi un total de cinq fichiers dans le répertoire "Plateforme".

```
“for i in {1..4}
do
  cp ~/Documents/Plateforme/mon_texte.txt
  ~/Documents/Plateforme/mon_texte$i.txt
done”
```

Ici j’ai créé une boucle pour que la créations des 4 fichiers “txt” se créent automatiquement et pour ne pas l’avoir à le faire manuellement.

For i in {1..4} permet de répéter la commande 4 fois.

Le cp permet de créer les 4 fichiers “txt” et le \$i c’est parce que i=1 puis 2,3,4.



Ensuite, archivez le répertoire "Plateforme" en utilisant les commandes "tar" et "gzip". Explorez différentes options de compression lors de cette étape.

```
“tar -cvf ~/Documents/Plateforme.tar ~/Documents/Plateforme”
```

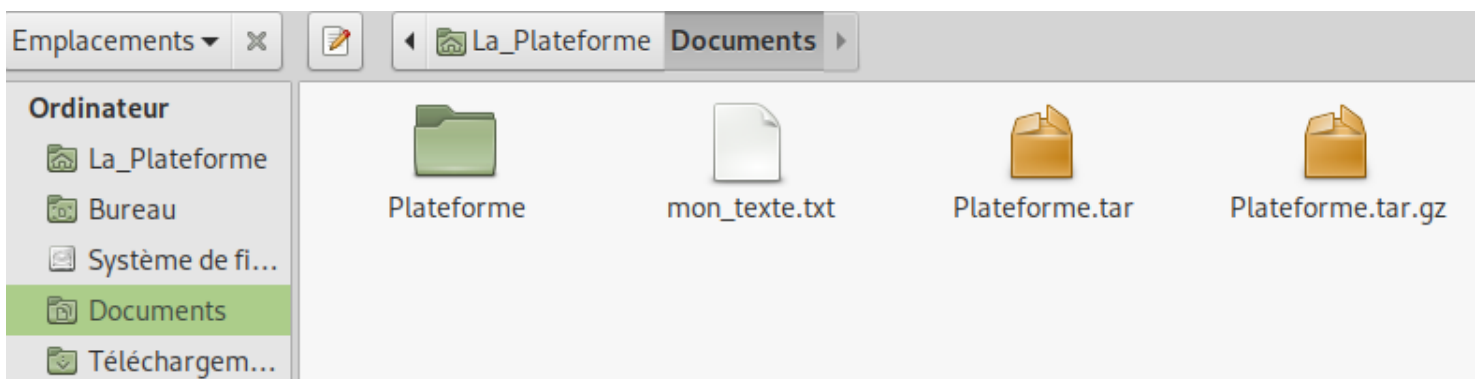
Ici on crée un dossier archivé “sans compression (.tar)”

Le “-cvf” = “-c” pour créer une archive, “-v” pour afficher le fichier archivé et “-f” c’est le fichier de sortie. Ensuite on a le chemin du dossier de base et le chemin où on veut le mettre.

```
“tar -czvf ~/Documents/Plateforme.tar.gz ~/Documents/Plateforme”
```

Ici on crée un dossier archivé “avec compression “gzip” (.tar.gz)”

On ajoute un “-z” au “-cvf” afin d’ajouter la compression avec “gzip”.



Il est aussi possible de compresser avec “gzip -9” c’est la compression maximale. Mais aussi “bzip2” qui est une meilleure compression que “gzip” simple mais qui est plus lente.

Décompressez les archives créées en utilisant les commandes appropriées. Explorez les diverses options de décompression.

```
“tar -xvf ~/Documents/Plateforme.tar -C ~/Documents/”
```

Ici la commande permet de décompresser l'archive "sans compression" ".tar".

Le "xvf" = "-x" pour l'extraction, le "-v" pour afficher le fichier, le "-f" qui est le fichier à extraire. Et "-C" pour indiquer le dossier de destination ici "Documents".

```
"tar -xzf ~/Documents/Plateforme.tar.gz -C ~/Documents/"
```

Ici la commande permet de décompresser l'archive "compressé gzip" ".tar.gz".

Ici on ajoute un "-z" au "xvf" pour gérer la compression "gzip". Tout le reste est pareil que pour le ".tar"

Pour commencer, créer un script python permettant la création d'un fichier CSV et l'ajout des données. Une fois votre script python créé et fonctionnel, extraire les informations relatives aux villes de chaque personne en utilisant la commande "awk".

```
GNU nano 7.2 create_csv.py
import csv

with open("personnes.csv", "w") as fichier:
    writer = csv.writer(fichier)
    writer.writerow(["Nom", "Âge", "Ville"])
    writer.writerow(["Jean", 25, "Paris"])
    writer.writerow(["Marie", 30, "Lyon"])
    writer.writerow(["Pierre", 22, "Marseille"])
    writer.writerow(["Sophie", 35, "Toulouse"])
```

Mon Script

Commandes script + execution
+ résultat

```
La_Plateforme@debian01:~$ nano create_csv.py
La_Plateforme@debian01:~$ python3 create_csv.py
La_Plateforme@debian01:~$ cat personnes.csv
Nom,Âge,Ville
Jean,25,Paris
Marie,30,Lyon
Pierre,22,Marseille
Sophie,35,Toulouse
La_Plateforme@debian01:~$
```

```
La_Plateforme@debian01:~$ awk -F"," 'NR > 1 { print $1 " habite à " $3 }' personnes.csv
Jean habite à Paris
Marie habite à Lyon
Pierre habite à Marseille
Sophie habite à Toulouse
```

Awk



create_csv.py



personnes.csv

Dans le but d’explorer les processus, commencez par recenser tous ceux qui sont actifs sur votre système. Cherchez la commande permettant de fermer un processus, puis exécutez-la pour terminer un processus spécifique.

Mettre en place une surveillance en temps réel de l’utilisation du CPU, de la mémoire et d’autres ressources système. Affichez les informations dans le terminal à l’aide de commandes appropriées.

“top” pour l’utilisation du cpu et de la mémoire.

“vmstat 1” afin d’afficher un résumé toutes les secondes du cpu, des statistiques mémoire, etc...

“watch -n 1 free -h” afin de mettre à jour toutes les seconde grâce au “-n 1”

Créez un script automatisant la recherche de mise à jour des logiciels existants sur le système. Il doit offrir la possibilité à l’utilisateur de procéder à la mise à jour de ces logiciels.

```
La_Plateforme@debian01: ~  
Fichier Édition Affichage Recherche Terminal Aide  
_a_Plateforme@debian01:~$ nano mise_a_jour.sh  
_a_Plateforme@debian01:~$ chmod +x mise_a_jour.sh  
_a_Plateforme@debian01:~$ ./mise_a_jour.sh  
Recherche de mises à jour disponibles...  
[sudo] Mot de passe de La_Plateforme :  
Désolé, essayez de nouveau.  
[sudo] Mot de passe de La_Plateforme :  
_a_Plateforme n'est pas dans le fichier sudoers.  
  
Voici la liste des paquets pouvant être mis à jour:  
En train de lister... Fait  
  
Voulez-vous procéder à la mise à jour maintenant ? (o/n) : o  
Mise à jour en cours...  
[sudo] Mot de passe de La_Plateforme :  
_a_Plateforme n'est pas dans le fichier sudoers.  
Mise à jour terminée.  
_a_Plateforme@debian01:~$  
  
#!/bin/bash  
  
echo " Recherche de mises à jour disponibles..."  
sudo apt update  
  
echo ""  
echo " Voici la liste des paquets pouvant être mis à jour:"  
apt list --upgradable  
  
echo ""  
read -p " Voulez-vous procéder à la mise à jour maintenant ? (o/n) : " choix  
  
if [[ "$choix" == "o" || "$choix" == "O" ]]; then  
    echo " Mise à jour en cours..."  
    sudo apt upgrade -y  
    echo " Mise à jour terminée."  
else  
    echo " Mise à jour annulée."  
fi
```

Élaborez un script ayant pour objectif de simplifier l'installation et la gestion des dépendances logicielles pour un projet web, tout assurant la compatibilité entre les différentes versions.

Le script doit installer les éléments suivants :

- Un serveur Web (Apache ou Ngnix)
- phpMyAdmin
- Un système de gestion de base de données relationnelle (MySQL ou MariaDB)
- Un environnement JavaScript côté serveur (Node.js) avec npm
- Un système de contrôle de version (git)

```
GNU nano 7.2                                install_web.sh
#!/bin/bash

# ===== CONFIGURATION =====
LOGFILE="web_install.log"
echo "Installation lancée le $(date)" > $LOGFILE

# ===== FONCTIONS =====
log() {
    echo "[+] $1" | tee -a $LOGFILE
}

error_exit() {
    echo "[ERREUR] $1" | tee -a $LOGFILE
    exit 1
}

# ===== MISE À JOUR =====
log "Mise à jour des paquets..."
apt update -y && apt upgrade -y || error_exit "Échec de la mise à jour"

# ===== INSTALLATION APACHE =====
log "Installation de Apache..."
apt install apache2 -y || error_exit "Apache non installé"

# ===== INSTALLATION MARIADB =====
log "Installation de MariaDB..."
apt install mariadb-server -y || error_exit "MariaDB non installé"

# ===== INSTALLATION PHP ET PHPMYADMIN =====
log "Installation de PHP et phpMyAdmin..."
apt install php libapache2-mod-php php-mysql php-cli php-curl php-zip php-mbstring php-xml php-common php-bcmath unzip -y || error_exit "PHP non installé"
apt install phpmyadmin -y || error_exit "phpMyAdmin non installé"
```

```
GNU nano 7.2                                install_web.sh
log "Installation de Apache..."
apt install apache2 -y || error_exit "Apache non installé"

# ===== INSTALLATION MARIADB =====
log "Installation de MariaDB..."
apt install mariadb-server -y || error_exit "MariaDB non installé"

# ===== INSTALLATION PHP ET PHPMYADMIN =====
log "Installation de PHP et phpMyAdmin..."
apt install php libapache2-mod-php php-mysql php-cli php-curl php-zip php-mbstring php-xml php-common php-bcmath unzip -y || error_exit "PHP non installé"
apt install phpmyadmin -y || error_exit "phpMyAdmin non installé"

# ===== INSTALLATION NODEJS ET NPM =====
log "Installation de Node.js et npm..."
apt install nodejs npm -y || error_exit "Node.js ou npm non installés"

# ===== INSTALLATION GIT =====
log "Installation de Git..."
apt install git -y || error_exit "Git non installé"

# ===== VÉRIFICATION DES VERSIONS =====
log "Vérification des versions installées..."
apache2 -v | tee -a $LOGFILE
mysql --version | tee -a $LOGFILE
php -v | tee -a $LOGFILE
node -v | tee -a $LOGFILE
npm -v | tee -a $LOGFILE
git --version | tee -a $LOGFILE

# ===== FIN =====
log "✓ Installation complète !"
```

À l'aide d'un Script Shell , exploiter les données d'une API Web (celle de votre choix). Assurez-vous que la communication avec l'API se fasse de manière sécurisée. Inclure une gestion des erreurs afin d'anticiper tout comportement inattendu en cas de problème avec l'API.

```
GNU nano 7.2 meteo_openweather.sh
#!/bin/bash

# Vérifie les dépendances
if ! command -v curl &> /dev/null; then
    echo "× curl n'est pas installé. Installez-le avec : apt install curl"
    exit 1
fi

if ! command -v jq &> /dev/null; then
    echo "× jq n'est pas installé. Installez-le avec : apt install jq"
    exit 1
fi

# === CONFIGURATION ===
API_KEY="c55bee959cab5b4bdf88475c2b24e542" # clé API personnelle
CITY="Cannes"
COUNTRY="FR"
API_URL="https://api.openweathermap.org/data/2.5/weather?q=${CITY},${COUNTRY}&appid=${API_KEY}&units=metric&lang=fr"

echo "📡 Récupération de la météo pour $CITY..."

# === APPEL API ===
RESPONSE=$(curl -s -w "%{http_code}" -o response.json "$API_URL")
HTTP_CODE="${RESPONSE: -3}"

# === GESTION ERREURS ===
if [ "$HTTP_CODE" != "200" ]; then
    echo "× Échec de la requête. Code HTTP : $HTTP_CODE"
    cat response.json | jq '.message' 2>/dev/null || echo "(erreur inconnue)"
    rm -f response.json
    exit 1
fi
```

```
# === GESTION ERREURS ===
if [ "$HTTP_CODE" != "200" ]; then
    echo "× Échec de la requête. Code HTTP : $HTTP_CODE"
    cat response.json | jq '.message' 2>/dev/null || echo "(erreur inconnue)"
    rm -f response.json
    exit 1
fi

# === EXTRACTION DES DONNÉES ===
echo "✅ Météo actuelle à $CITY :"
echo "-----"
jq '.weather[0].description' response.json
jq '.main.temp' response.json | xargs -I {} echo "Température : {} °C"
jq '.main.humidity' response.json | xargs -I {} echo "Humidité : {} %"
jq '.wind.speed' response.json | xargs -I {} echo "Vent : {} m/s"
echo "-----"

# Nettoyage
rm -f response.json
```